

```
In [319... import qutip as qt
from qutip import Qobj
import numpy as np

# Define polarization states
H = Qobj([[1], [0]]) # Horizontal polarization
A = (1 / np.sqrt(2)) * Qobj([[1], [-1]]) # Anti-diagonal polarization
V = Qobj([[0], [1]]) # Vertical polarization
D = (1 / np.sqrt(2)) * Qobj([[1], [1]]) # Diagonal polarization

num_send = 100 # the number of photons Alice send
```

```
In [320... def ADC(qobj):
    """
    translate qobj to char variable, so that the public channel function is

    Parameter(s):
        qobj: the qobj needed to be translated

    Return(s):
        the corresponding character to the input.
    """
    if qobj==H:
        return "H"
    if qobj==V:
        return "V"
    if qobj==D:
        return "D"
    if qobj==A:
        return "A"
```

part a).

The following box belongs to Alice

```
In [321... def Alice(num_photon):
    """
    "Alice" is a function that accepts an integer num_photon which is the nu

    Parameter(s):
        num_photon: the number of photon

    Return(s):
        a list that has two columns, the first column is the index(starting
    """
    return [(i+1, np.random.choice([H, A])) for i in range(num_photon)]
```

part b).

The following box belongs to Bob

```
In [322... def Bob(photon_list):
    """
    "Bob" accepts the output of "Alice" and returns only the events where he
    Parameter(s):
        photon_list: the output of Alice, a list that consist of many tuple
    Return(s):
        received_message: the output of Bob, photons that Bob get a pin and
    """
    received_message=[]
    for index, photon in photon_list:
        state = 'A' if photon == A else 'H'
        receiver = np.random.choice(['V','D'])
        if receiver == 'V' and state == 'H' or receiver == 'D' and state == 'A':
            pass # No pin Possible
        elif receiver == 'D' and state == 'H':
            pin = np.random.choice(['H', 'Null']) # simulate the process of
            if pin != 'Null':
                received_message.append((index, pin)) # register the pin
        elif receiver == 'V' and state == 'A': # similar to the last elif s
            pin = np.random.choice(['A', 'Null'])
            if pin != 'Null':
                received_message.append((index, pin))
    return received_message
```

part c). is two boxes down

part d).

```
In [323... def Eve(photon_list):
    """
    a spy "Eve" who takes the output that "Alice" produces, performs the sam
    Parameter(s):
        photon_list: the output of Eve, a list that consist of many tuple of
    Return(s):
        received_message: the output of Bob, photons that Bob get a pin and
    """
    received_message=[]
    for index, photon in photon_list:
        state = 'A' if photon == A else 'H'
        receiver = np.random.choice(['V','D'])
        if receiver == 'V' and state == 'H' or receiver == 'D' and state == 'A':
            received_message.append((index, receiver)) # indecate the detec
        elif receiver == 'D' and state == 'H':
            pin = np.random.choice(['H', receiver]) # simulate the process
            received_message.append((index, pin))
        elif receiver == 'V' and state == 'A': # similar to the last elif s
            pin = np.random.choice(['A', receiver])
```

```

        received_message.append((index, pin))

    forged_message = []

    for index, click in received_message:
        if click == "V":
            forged_message.append((index, H))
        elif click == "D":
            forged_message.append((index, A))
        elif click == "H":
            forged_message.append((index, H))
        elif click == "A":
            forged_message.append((index, A))

    return forged_message

```

In [324...

```

def public_channel(click_received, photon_sent):
    """
    This channel is used to varify the click Bob got with the photon Alice s

    Parameter(s):
        click_received: a part of the click bob received

    Return(s):
        check_list: a list that contains for each click Bob claim he receive
        true_rate: the proportion of clicks that Bob got it right.
    """
    check_list = []
    for i, t in enumerate(click_received) :
        index, click = t
        if ADC(photon_sent[index - 1][1]) == click:
            check_list.append((index, True))
        else:
            check_list.append((index, False))
    # calculate the ratio of correct count to the entire compared clicks
    true_count = 0
    for i, b in check_list:
        if b:
            true_count += 1
    true_rate = true_count/len(check_list)
    error_rate = 1-true_rate
    return check_list, error_rate

```

part c).

In [325...

```

photon_sent = Alice(num_send) # the photons Alice sent
click_received = Bob(photon_sent[0:int(len(photon_sent)/2)]) # the click Bo
temp, rate = public_channel(click_received, photon_sent)
print("Error rate: ", rate)

```

Error rate: 0.0

part e).

the sample number 100 is too small to give a result that reflects Eve's smartness/dumbness, statistics are shown in part f).

```
In [335... photon_sent = Alice(num_send) # the photons Alice sent
photon_sent_E = Eve(photon_sent) # Big brother(big sister in this case) is
click_received = Bob(photon_sent_E[0:int(len(photon_sent_E)/2)]) # the clic
temp, rate = public_channel(click_received, photon_sent)
print('Error rate: ', rate)
```

Error rate: 0.3846153846153846

part f).

```
In [327... rate_list = []
for i in range(10000):
    photon_sent = Alice(num_send) # the photons Alice sent
    photon_sent_E = Eve(photon_sent)
    click_received = Bob(photon_sent_E[0:int(len(photon_sent_E)/2)]) # the
    temp, rate = public_channel(click_received, photon_sent)
    rate_list.append(rate)

print('Error rate: ', np.mean(rate_list), ' Variance: ', np.var(rate_list))
```

Error rate: 0.25155174354421694 Variance: 0.016091207190272384

```
In [328... # length_list = []
# for i in range(10000):
#     photon_sent = Alice(num_send) # the photons Alice sent
#     photon_sent_E = Eve(photon_sent)
#     click_received = Bob(photon_sent_E[0:int(len(photon_sent_E)/2)]) # th
#     temp, rate = public_channel(click_received, photon_sent)
#     length_list.append(len(temp))

# print('Mean length with Eve: ', np.mean(length_list), ' Variance: ', np.var(
```

```
In [329... # length_list = []
# for i in range(10000):
#     photon_sent = Alice(num_send) # the photons Alice sent
#     # photon_sent_E = Eve(photon_sent)
#     click_received = Bob(photon_sent[0:int(len(photon_sent)/2)]) # the cl
#     temp, rate = public_channel(click_received, photon_sent)
#     length_list.append(len(temp))

# print('Mean length without Eve: ', np.mean(length_list), ' Variance: ', np.
```